Linear regression finds the coefficients of a function which minimize the error

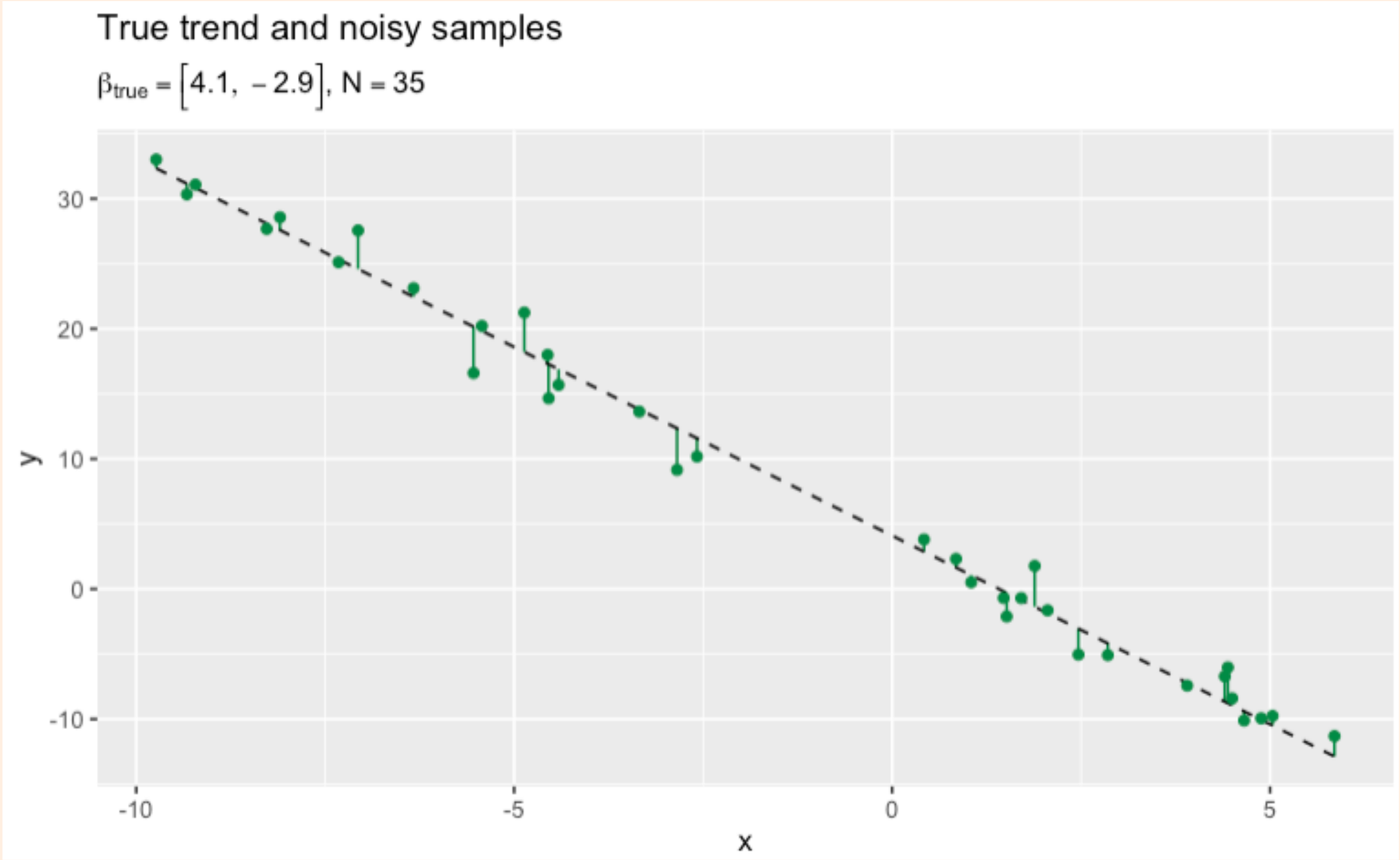
- We want to train or fit a model between the response y and the input x
- We will start out with a simple linear relationship

$$y = \beta_0 + \beta_1 x + \text{error}$$

- The task of machine learning is to estimate the values of the parameters from the data by minimizing the observed error between the model's predictions and the true labels

Another synthetic dataset

Simple linear relationship, fairly low noise



Linear regression components vector notation

for a single-input, linear model with only slope and intercept parameters

Input: $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \cdots \\ x_n \end{bmatrix}$

Response: $\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \cdots \\ y_n \end{bmatrix}$

Coefficients: $\boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix}$

Prediction:

$$\boldsymbol{\mu} = \begin{bmatrix} \beta_0 + \beta_1 x_1 \\ \beta_0 + \beta_1 x_2 \\ \cdots \\ \beta_0 + \beta_1 x_n \end{bmatrix}$$

Objective/error/loss function

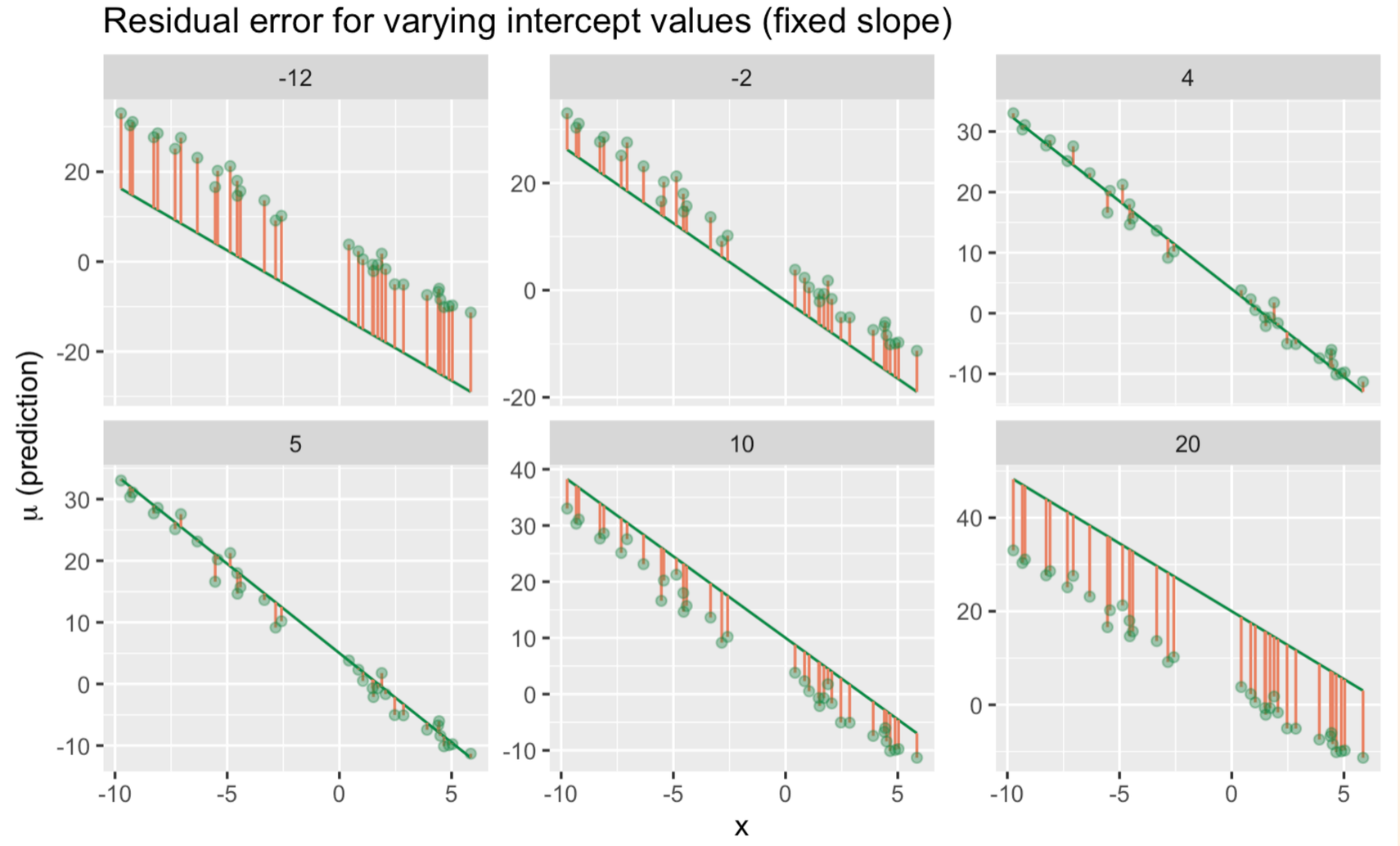
Total deviance from true labels (SSE for linear regression)

$$L = \sum_{n=1}^{N} (y_n - \mu_n)^2$$

$$= \sum_{n=1}^{N} (y_n - (\beta_0 + \beta_1 x_n))^2$$

Choose coefficients that lead to minimum error on the training set

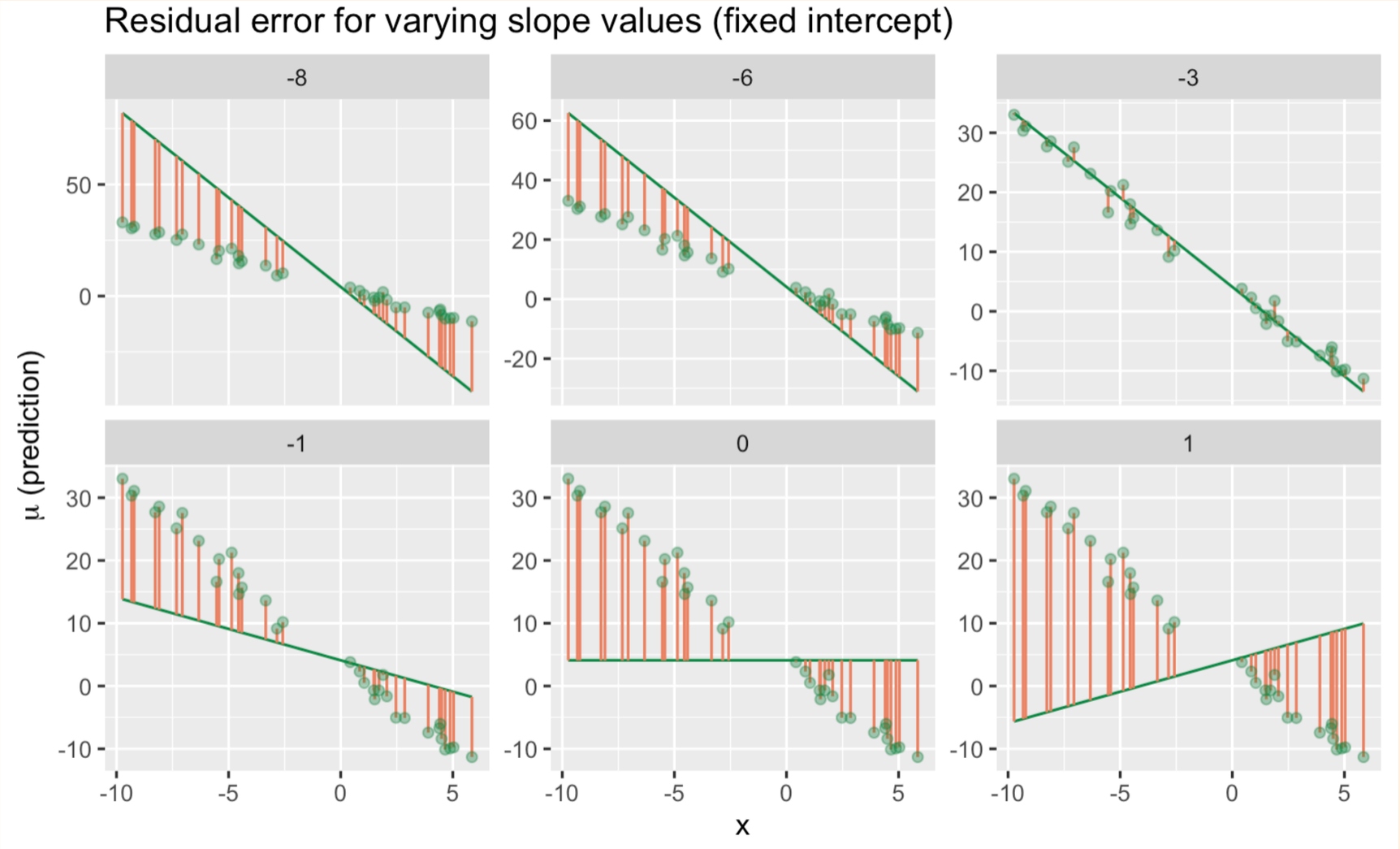
adjusting intercept, β_0, moves the predicted trend up and down



orange bars show "residuals", or differences between predicted and true labels

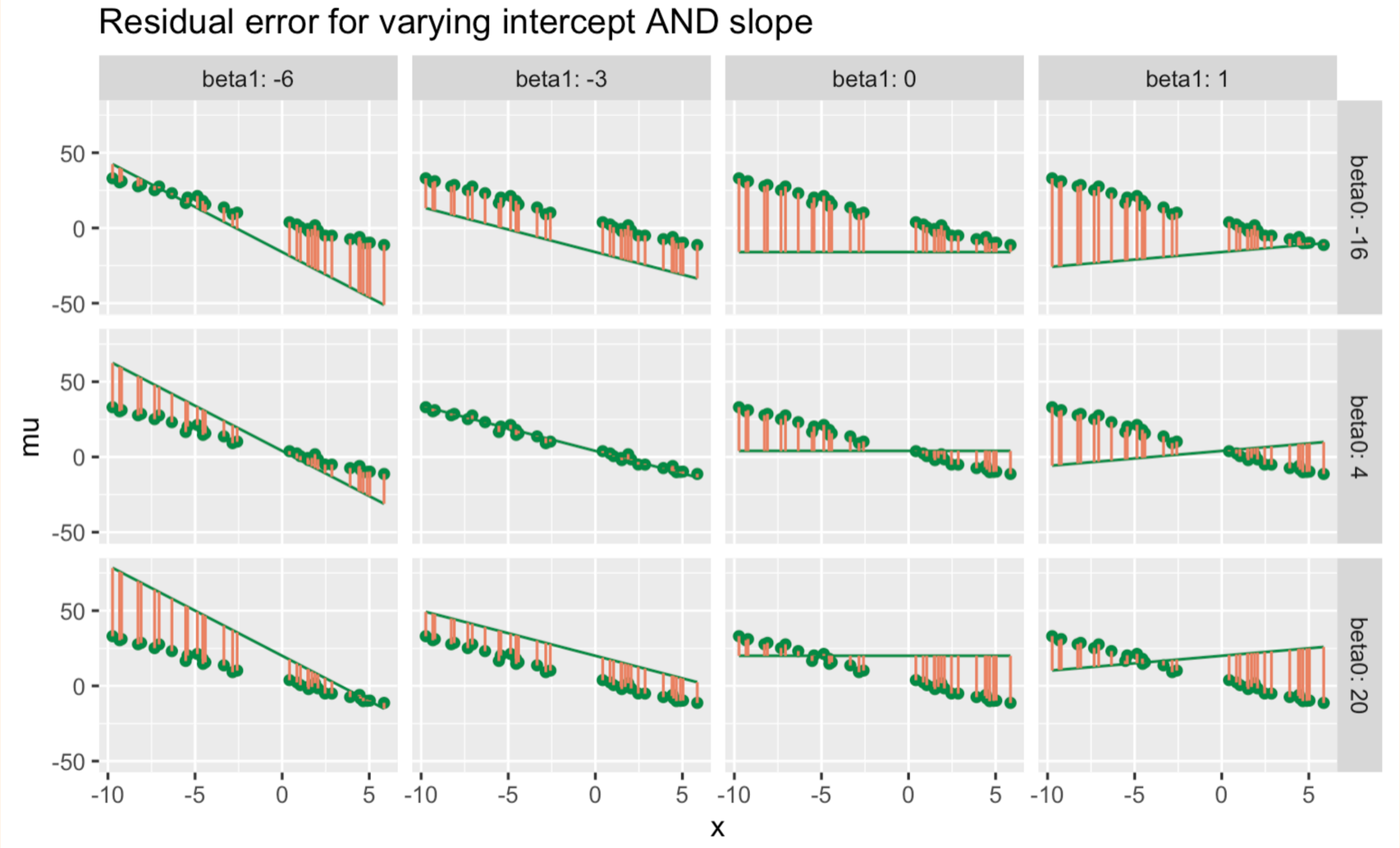
Choose coefficients that lead to minimum error on the training set

adjusting slope, β_1, rotates the predicted trend line



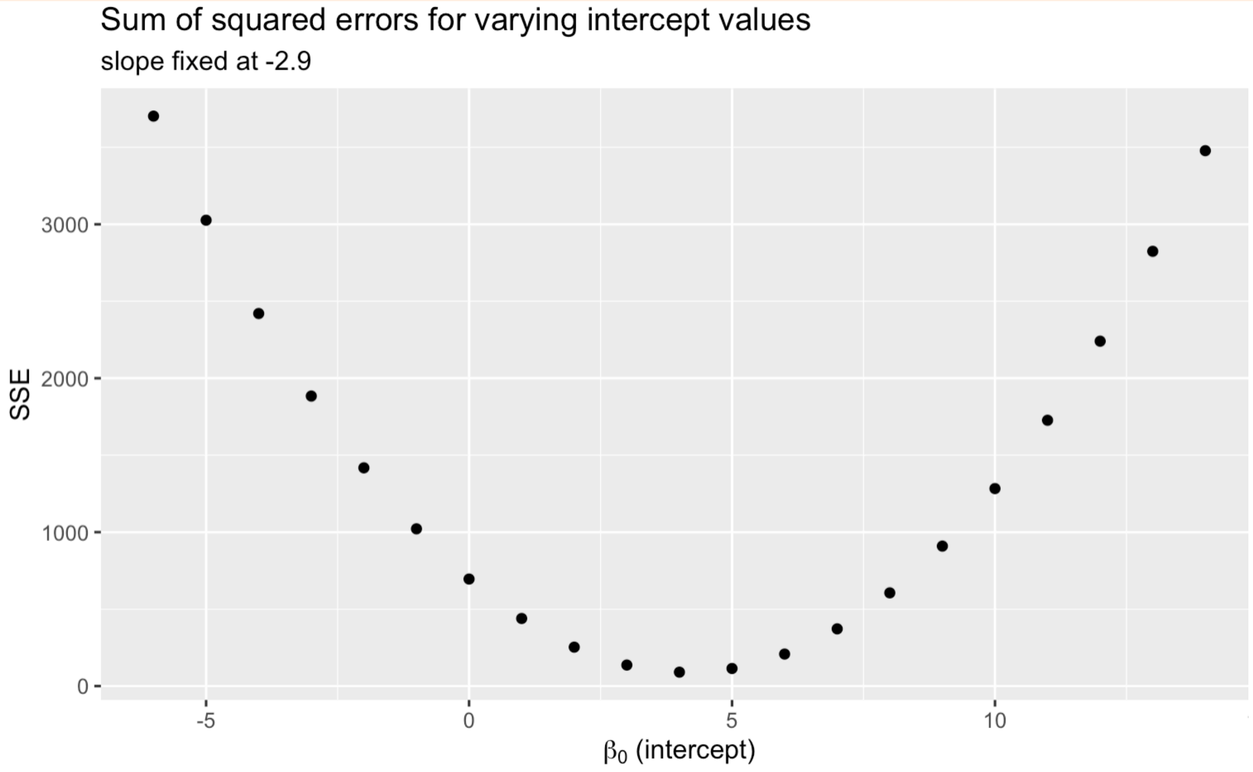
Must find the combination of (β_0, β_1) that leads to the overall minimum error

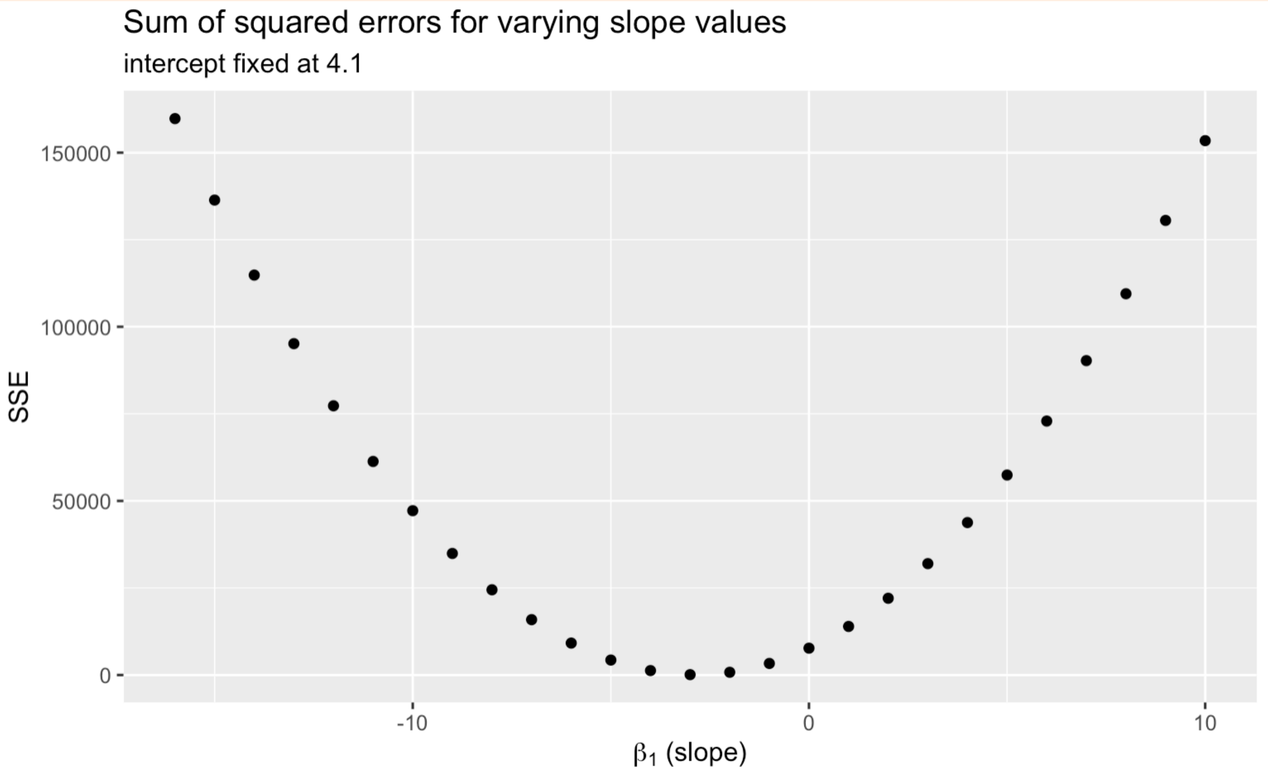
graph below varies both parameters



Must find the combination of (β_0, β_1) that leads to the overall minimum error

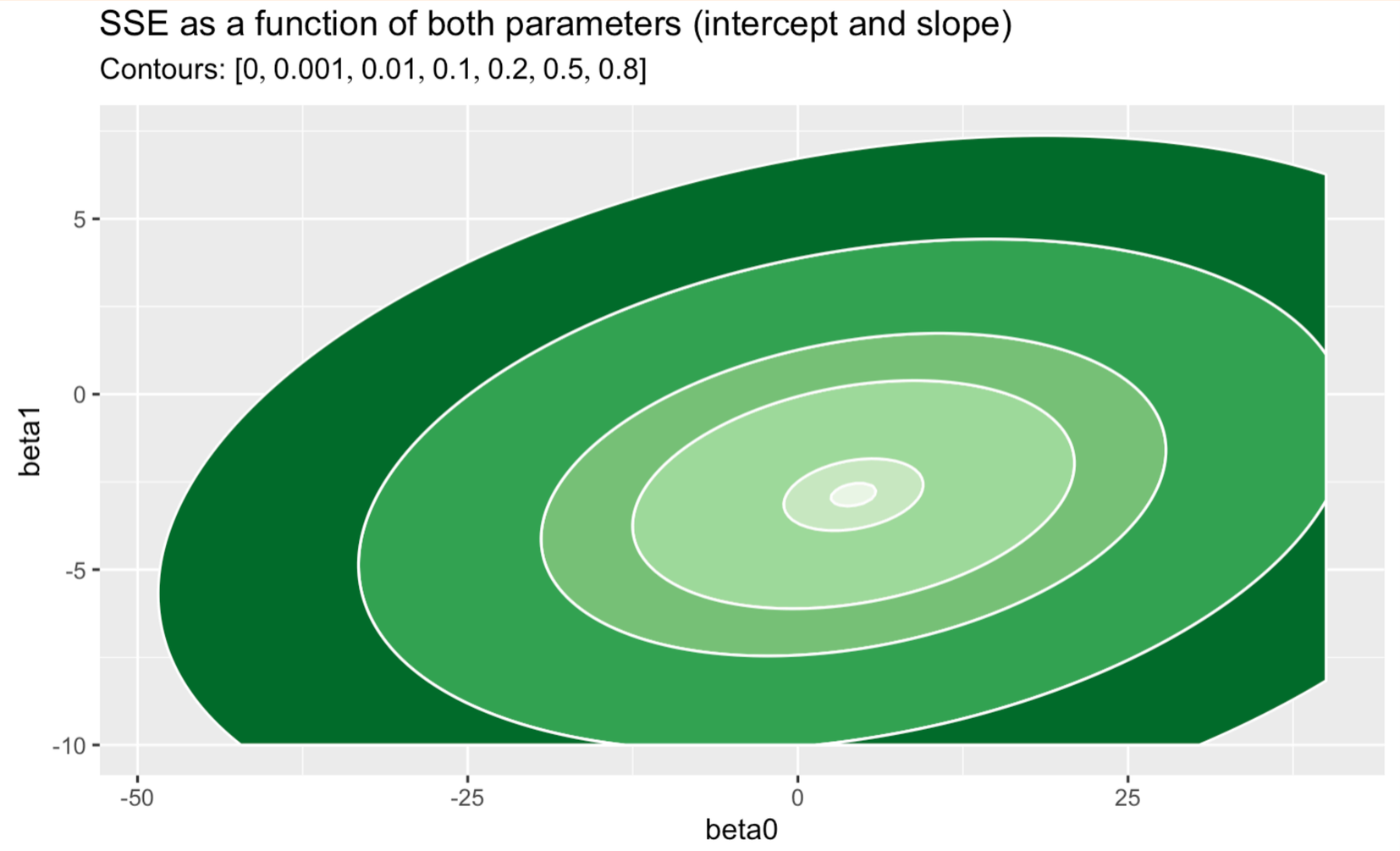
below: sum all squared residuals for different estimates of the parameters





Must find the combination of (β_0, β_1) that leads to the overall minimum error

surface/contour plot of SSE for different parameter combinations



fill color represents SSE (greener=higher) for combination of intercept (x-axis) and slope (y-axis)

contours show quantiles of SSE

e.g. center white circle is 0.001, or bottom 99.9th percentile

Use optimization to choose best coefficients

find the value of a variable(s) that maximizes/minimizes some function

- Define the objective function (i.e. SSE)
- Determine which variables are fixed (inputs, outputs, number of parameters, basis function), and which are tuneable (coefficient values)
- Include any constraints on objective (discussed in-depth later)
- Maximize/minimize the objective function by calculating the derivative with respect to each of the β parameters, setting equal to zero, and solving for the unknown coefficients
  - *in some cases, an exact solution is not possible and we use an approximation instead.
- If the objective function is unimodal, there will exist exactly one optimal combination of coefficient values

Use optimization to choose best coefficients

find the value of a variable(s) that maximizes/minimizes some function

$$\hat{\boldsymbol{\beta}} = \begin{bmatrix} \hat{\beta}_0 \\ \hat{\beta}_1 \end{bmatrix} = \underset{(\beta_0, \beta_1)}{\operatorname{argmin}} \sum_{n=1}^{N} (y_n - (\beta_0 + \beta_1 x_n))^2$$

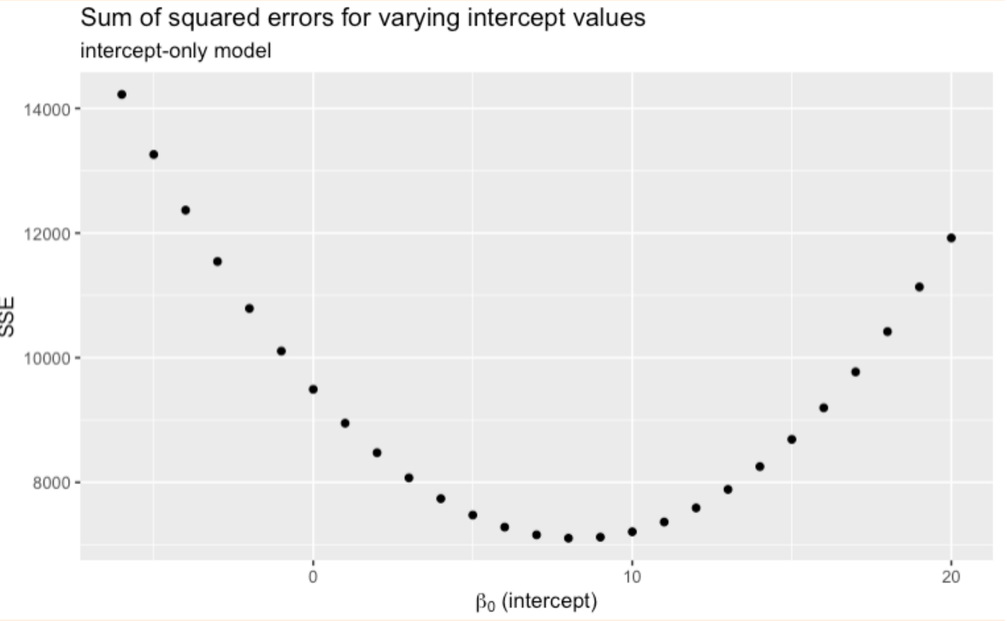
Ordinary least squares (OLS) estimator: select coefficients that minimize the sum of squared errors (SSE) on the training set

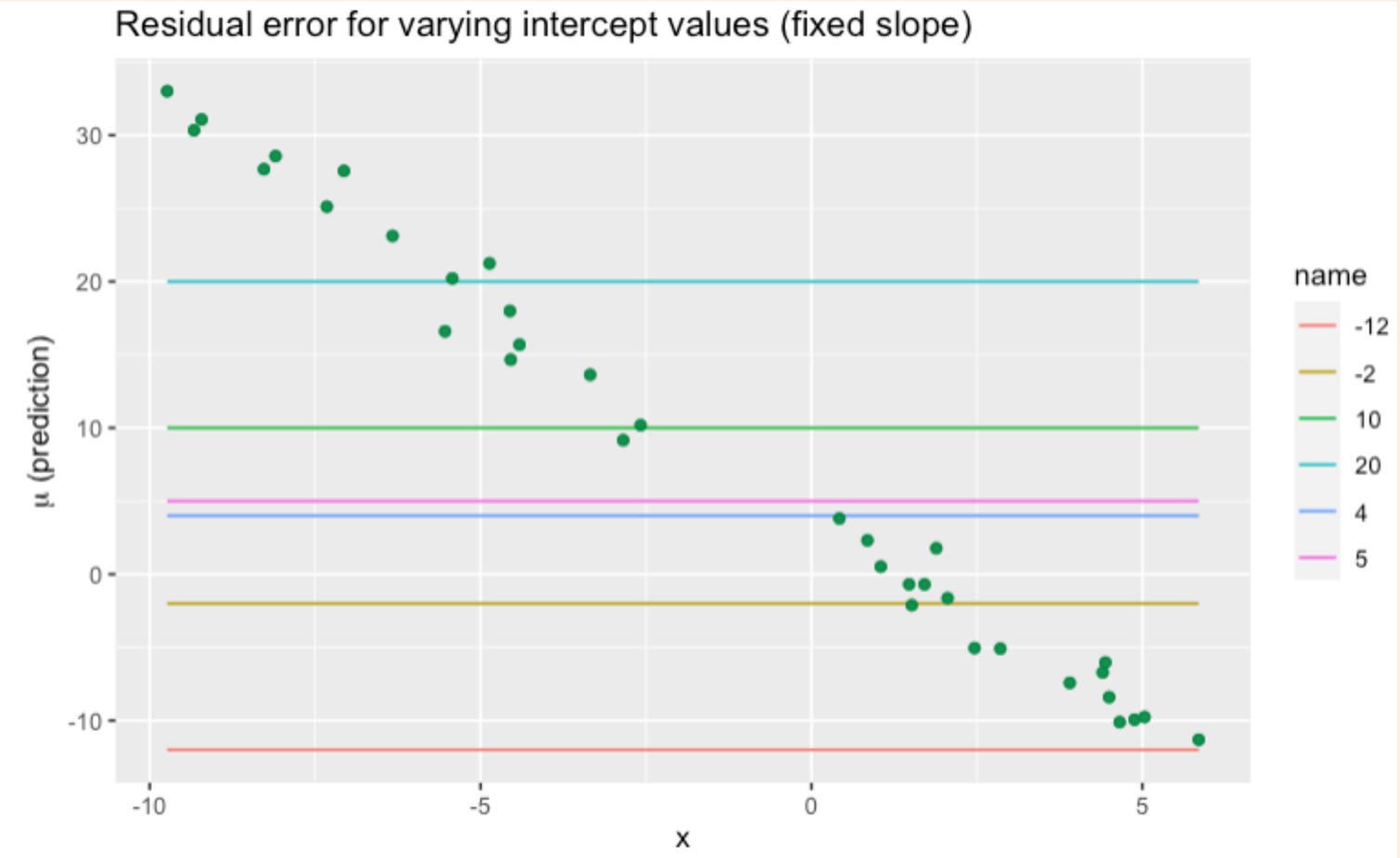
Simplify even further for now: intercept-only model

what is the best constant-only model?

$$\hat{\beta}_0 = \underset{\beta_0}{\operatorname{argmin}} \sum_{n=1}^{N} (y_n - \mu_n)^2$$

$$\mu_n = \beta_0$$





Calculate derivative of loss function with respect to intercept

$$L = SSE = \sum_{n=1}^{N} (y_n - \mu_n)^2$$

$$\frac{dL}{d\beta_0} = \frac{d}{d\beta_0} \sum_{n=1}^{N} (y_n - \beta_0)^2$$

$$= \sum_{n=1}^{N} \frac{d}{d\beta_0} (y_n - \beta_0)^2$$

$$= \sum_{n=1}^{N} 2(y_n - \beta_0) \frac{d}{d\beta_0} (y_n - \beta_0)$$

$$= \sum_{n=1}^{N} 2(y_n - \beta_0)[-1]$$

$$\cdots$$

$$\cdots$$

$$= -2 \sum_{n=1}^{N} (y_n - \beta_0)$$

$$= -2 \left[\sum_{n} y_n - \sum_{n} \beta_0 \right]$$

$$= -2 \left[N\bar{y} - N\beta_0 \right]$$

$$= -2N \left[\bar{y} - \beta_0 \right]$$

Set derivative equal to zero and solve for β_0

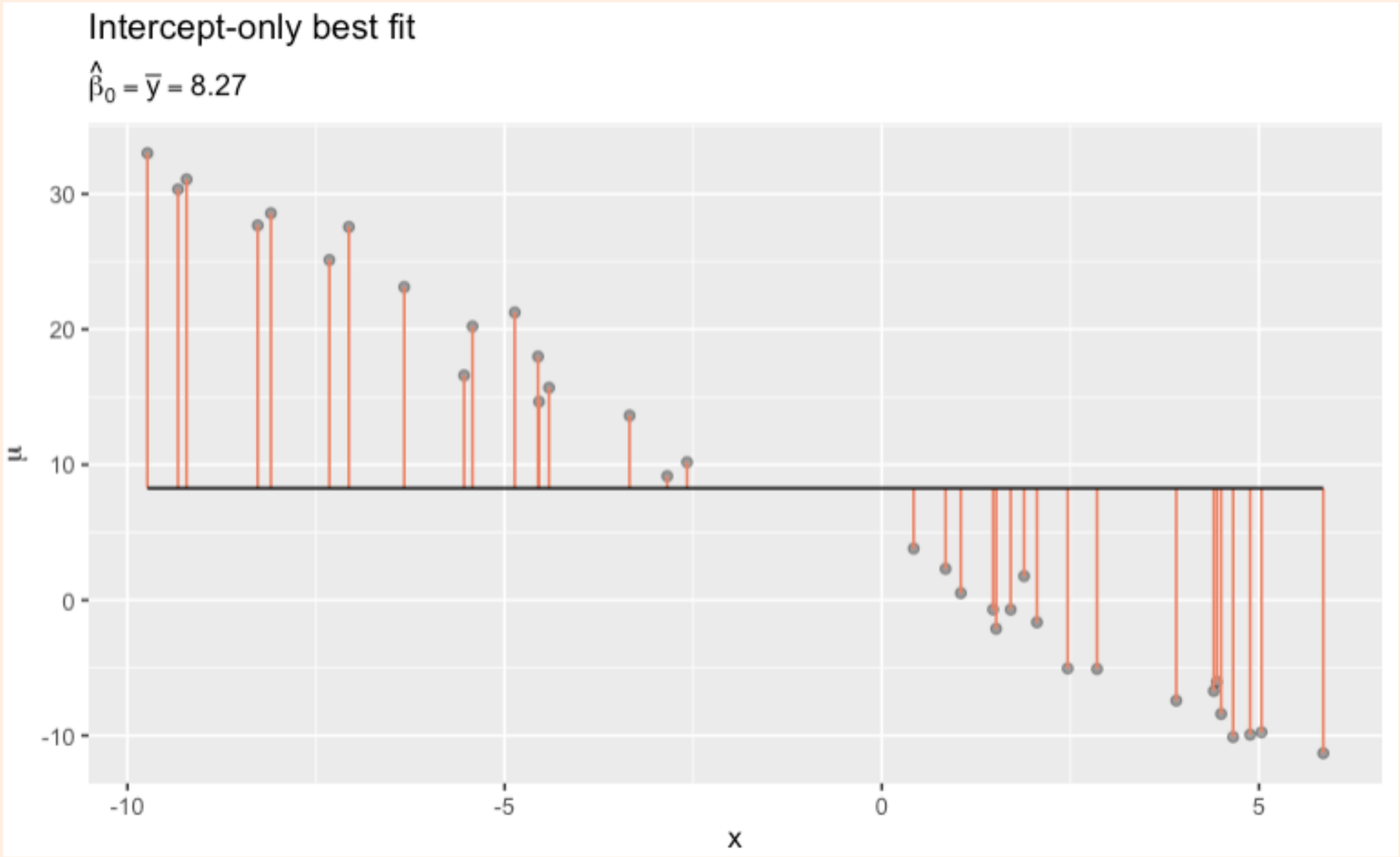
the best value for the intercept is the average response!

$$0 = -2N\left[\bar{y} - \beta_0\right]$$

$$0 = \left[\bar{y} - \beta_0\right]$$

$$\hat{\beta}_0 = \bar{y}$$

$$\hat{\beta}_0 = \frac{1}{N}\sum_{n=1}^{N} y_n$$



Verify that the solution is a local minimum

Second derivative test

- Since the roots of a function may correspond to maxima, minima, or saddle points, we should check the second derivative to ensure we choose the value that **minimizes** the SSE

$$\frac{\partial^2 L}{\partial \beta^2} < 0 \quad \hat{\beta} \text{ is a local max.}$$

$$\frac{\partial^2 L}{\partial \beta^2} > 0 \quad \hat{\beta} \text{ is a local min.}$$

$$\frac{\partial^2 L}{\partial \beta^2} = 0 \quad \hat{\beta} \text{ is a saddle point}$$

$$\frac{d^2 L}{d\beta_0^2} = \frac{d}{d\beta_0}\left(-2N\left[\bar{y} - \beta_0\right]\right)$$

$$= -2N\frac{d}{d\beta_0}\left[\bar{y} - \beta_0\right]$$

$$= 2N$$

Since N (sample size) is always positive, the second derivative of the objective function is also positive. Therefore, it is a local minimum

Solving for multiple unknown parameters

Intercept-only model is not very useful…how do we find the solution when there is more than one parameter?

Need the gradient vector

- Calculate the partial derivative of the loss function with respect to each one of the parameters individually. For partial derivatives, all variables that are not being differentiated by are treated as constants
- The gradient vector is a column vector with one row for every unknown parameter.
- Each entry in the gradient vector represents the first partial derivative of the loss function with respect to one of the unknown coefficients.

$$\mathbf{g} = \nabla L = \begin{bmatrix} \frac{\partial L}{\partial \beta_0} \\ \frac{\partial L}{\partial \beta_1} \\ \vdots \\ \frac{\partial L}{\partial \beta_D} \end{bmatrix}$$

Return to the simple linear model

$$\mu(x_n) = \beta_0 + \beta_1 x_n$$

- There are now two coefficients we wish to estimate
- We will need to calculate the partial derivatives with respect to both coefficients

$$\mathbf{g} = \nabla L = \begin{bmatrix} \frac{\partial L}{\partial \beta_0} \\ \frac{\partial L}{\partial \beta_1} \end{bmatrix}$$

Find OLS estimate for the intercept parameter

Calculate derivative wrt. β_0

$$L = \sum_{n=1}^{N} (y_n - (\beta_0 + \beta_1 x_n))^2$$

$$\frac{\partial L}{\partial \beta_0} = \frac{\partial}{\partial \beta_0} \sum_{n=1}^{N} (y_n - (\beta_0 + \beta_1 x_n))^2$$

$$= \sum_{n=1}^{N} \frac{\partial}{\partial \beta_0} (y_n - (\beta_0 + \beta_1 x_n))^2$$

$$= \sum_{n=1}^{N} 2(y_n - (\beta_0 + \beta_1 x_n)) \frac{\partial}{\partial \beta_0} (y_n - \beta_0 - \beta_1 x_n)$$

$$= 2 \sum_{n=1}^{N} (y_n - \beta_0 - \beta_1 x_n)(-1)$$

Set equal to zero and solve for β_0

$$0 = -2 \sum_{n=1}^{N} (y_n - \beta_0 - \beta_1 x_n)$$

$$0 = \sum_{n} y_n - \sum_{n} \beta_0 - \beta_1 \sum_{n} x_n$$

$$0 = N\bar{y} - N\beta_0 - N\beta_1 \bar{x}_n$$

$$\hat{\beta}_0 = \bar{y} - \beta_1 \bar{x}_n$$

Find OLS estimate for the slope parameter

Calculate derivative wrt. β_1

$$L = \sum_{n=1}^{N} (y_n - (\beta_0 + \beta_1 x_n))^2$$

$$\frac{\partial L}{\partial \beta_1} = \frac{\partial}{\partial \beta_1} \sum_{n=1}^{N} (y_n - (\beta_0 + \beta_1 x_n))^2$$

$$= \sum_{n=1}^{N} \frac{\partial}{\partial \beta_1} (y_n - (\beta_0 + \beta_1 x_n))^2$$

$$= \sum_{n=1}^{N} 2(y_n - (\beta_0 + \beta_1 x_n)) \frac{\partial}{\partial \beta_1} (y_n - \beta_0 - \beta_1 x_n)$$

$$= 2 \sum_{n=1}^{N} (y_n - (\beta_0 + \beta_1 x_n))(-x_n)$$

Set equal to zero and solve for β_1

$$0 = -2 \sum_{n=1}^{N} (y_n x_n - \beta_0 x_n - \beta_1 x_n^2)$$

$$0 = \sum_{n=1}^{N} \left(y_n x_n - (\bar{y} - \beta_1 \bar{x}) x_n - \beta_1 x_n^2\right)$$

$$0 = \sum_{n=1}^{N} \left(y_n x_n - \bar{y} x_n + \beta_1 \bar{x} x_n - \beta_1 x_n^2\right)$$

$$0 = \sum_{n=1}^{N} \left(y_n x_n - \bar{y} x_n - \beta_1 \left[x_n^2 - \bar{x} x_n\right]\right)$$

$$0 = \sum_{n} (y_n x_n) - \bar{y} \sum_{n} x_n - \beta_1 \sum_{n} \left(x_n \left[x_n - \bar{x}\right]\right)$$

$$\beta_1 \sum_{n} \left(x_n \left[x_n - \bar{x}\right]\right) = \sum_{n} y_n x_n - N \bar{y} \bar{x}$$

$$\hat{\beta}_1 = \frac{\sum_n y_n x_n - N \bar{y} \bar{x}}{\sum_n \left(x_n \left[x_n - \bar{x}\right]\right)}$$

Let’s look more closely at the solution for the best slope

$$\hat{\beta}_1 = \frac{\sum_n y_n x_n - N\bar{y}\bar{x}}{\sum_n \left(x_n \left[x_n - \bar{x} \right] \right)}$$

Matrix multiplication

The most important thing for this class is to keep track of the dimensions of matrices being multiplied

To multiply two vectors/matrices, A and B, their inner dimensions much match

$$\begin{bmatrix} 2 & 4 & 3 \\ 2 & 8 & 7 \\ 1 & 4 & 5 \end{bmatrix} \begin{bmatrix} 2 & 4 \\ 2 & 8 \\ 1 & 4 \end{bmatrix}$$

[3x3][3x2] = [3x2]

The number of columns in A must match the rows in B

INVALID

$$\begin{bmatrix} 2 & 4 & 3 \\ 2 & 8 & 7 \\ 1 & 4 & 5 \end{bmatrix} \begin{bmatrix} 2 & 4 & 3 \\ 2 & 8 & 7 \end{bmatrix}$$

[3x3][2x3] = BAD

The product AB will have the same number of rows as A, and columns as B

Inner products (dot-products)

sum the products between paired elements in two vectors

$$\mathbf{a} = \begin{bmatrix} 3 \\ -9 \\ 4 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} -2 \\ 6 \\ -1 \end{bmatrix}$$

$$\mathbf{a}^T\mathbf{b} = \mathbf{b}^T\mathbf{a} = 3(-2) + (-9)6 + 4(1) = -56$$

Outer products

computes all pair-wise products between elements in two vectors

$$\mathbf{a} = \begin{bmatrix} 3 \\ -9 \\ 4 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} -2 \\ 6 \\ -1 \end{bmatrix}$$

$$\mathbf{a}\mathbf{b}^T = \begin{bmatrix} 3 \\ -9 \\ 4 \end{bmatrix} \begin{bmatrix} -2 & 6 & 1 \end{bmatrix} = \begin{bmatrix} 3(-2) & 3(6) & 3(1) \\ -9(-2) & -9(6) & -9(1) \\ 4(-2) & 4(6) & 4(1) \end{bmatrix}$$

$$= \begin{bmatrix} -6 & 18 & 3 \\ 18 & -54 & -9 \\ -8 & 24 & 4 \end{bmatrix}$$

Walk through an example of matrix multiplication

$$\mathbf{A} = \begin{bmatrix} 7 & -4 & 13 \\ 12 & -8 & 7 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 2 & 4 & 3 \\ 2 & 8 & 7 \\ 1 & 4 & 5 \end{bmatrix}$$

$$\mathbf{AB} = \begin{bmatrix} 19 & \blacksquare & \blacksquare \\ \blacksquare & \blacksquare & \blacksquare \end{bmatrix}$$

$$\mathbf{AB}_{1,1} = \sum_{d=1}^{3} \mathbf{A}_{1,d}\mathbf{B}_{d,1} = \mathbf{A}_{1,:}\mathbf{B}_{:,1} = 19$$

$$\mathbf{AB}_{1,2} = \sum_{d=1}^{3} \mathbf{A}_{1,d}\mathbf{B}_{d,2} = \mathbf{A}_{1,:}\mathbf{B}_{:,2} = 48$$

$$\mathbf{AB}_{1,3} = \sum_{d=1}^{3} \mathbf{A}_{1,d}\mathbf{B}_{d,3} = \mathbf{A}_{1,:}\mathbf{B}_{:,3} = 58$$

$$\mathbf{AB}_{2,1} = \sum_{d=1}^{3} \mathbf{A}_{2,d}\mathbf{B}_{d,1} = \mathbf{A}_{2,:}\mathbf{B}_{:,1} = 15$$

$$\mathbf{AB}_{2,2} = \sum_{d=1}^{3} \mathbf{A}_{2,d}\mathbf{B}_{d,2} = \mathbf{A}_{2,:}\mathbf{B}_{:,2} = 12$$

$$\mathbf{AB}_{2,3} = \sum_{d=1}^{3} \mathbf{A}_{2,d}\mathbf{B}_{d,3} = \mathbf{A}_{2,:}\mathbf{B}_{:,3} = 15$$

Calculate all vector products (or dot-product) between the ROWS of matrix A and the COLUMNS of matrix B

How could you rewrite this summation using matrix math?

$$\sum_{n=1}^{N} y_n x_n$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

Design matrix

augment data with a “fake” column for the intercept to simplify the expression for the prediction μ

$$\widehat{\mathbf{X}}^{N\times 2} = \begin{matrix} x_0 & x_1 \end{matrix} \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix} \quad \boldsymbol{\beta}^{2\times 1} = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix}$$

Single prediction

$$\begin{aligned} \mu_n &= \beta_0 + \beta_1 x_{n,1} \\ &= \beta_0 x_{n,0} + \beta_1 x_{n,1} \\ &= \sum_{d=0}^{1} \beta_d x_{n,d} = \mathbf{x}_{n,:}\, \boldsymbol{\beta} \end{aligned}$$

$d=0$ parameter index - 0 is always the intercept

All predictions at once

$$\boldsymbol{\mu}^{n\times 1} = \widehat{\mathbf{X}}\boldsymbol{\beta}$$

Design matrix

create in R with `model.matrix()`

```{r}
model.matrix(y ~ x, data = df)
```

```
   (Intercept)          x
1            1  2.8534551
2            1 -4.4118101
3            1  2.0568733
4            1  4.6572246
5            1 -4.5572988
```

Equivalent

```{r}
X_design <- model.matrix(y ~ x, data = df)
betas <- lm(y ~ x, data = df) |> pluck("coefficients")

lm_preds <- predict(lm(y ~ x, data = df), newdata = df)
matrix_math_preds <- X_design %*% betas

sum(lm_preds == matrix_math_preds)

```

```
[1] 35
```

In linear regression, predictions are made by simply taking the matrix product between the data and the learned parameters

Rewrite SSE objective with matrix math

replace summations with products

$$L = \sum_{n=1}^{N} (y_n - (\beta_0 x_{n,0} + \beta_1 x_{n,1}))^2$$

$$= \sum_{n=1}^{N} (y_n - \mathbf{x}_{n,:}\boldsymbol{\beta})^2$$

$$= (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$$

$$\begin{bmatrix}\delta_1 & \delta_2 & \dots & \delta_n\end{bmatrix} \begin{bmatrix}\delta_1 \\ \delta_2 \\ \vdots \\ \delta_n\end{bmatrix}$$

where $\delta_n = y_n - \mathbf{X}_{n,:}\boldsymbol{\beta}$

Derive the gradient vector

express all partial derivatives with a single matrix math equation!

$$L = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$$

$$\mathbf{g} = \nabla L = 2(-\mathbf{X}^T)(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$$

$$= -2(\mathbf{X}^T\mathbf{y} - \mathbf{X}^T\mathbf{X}\boldsymbol{\beta})$$

$[d \times n][n \times 1] = [d \times 1]$ $[d \times n][n \times d][d \times 1] = [d \times 1]$

You won't be required to do complex matrix math in this course. However, the Matrix Cookbook is an excellent resource for basic identities and theorems when you need to understand or simplify matrix expressions

Set the gradient equal to zero, solve for β

solve for all parameters in a single expression

$$0 = -2(\mathbf{X}^T\mathbf{y} - \mathbf{X}^T\mathbf{X}\boldsymbol{\beta})$$

$$\mathbf{X}^T\mathbf{X}\boldsymbol{\beta} = \mathbf{X}^T\mathbf{y}$$

$$\left[\mathbf{X}^T\mathbf{X}\right]^{-1}\mathbf{X}^T\mathbf{X}\boldsymbol{\beta} = \left[\mathbf{X}^T\mathbf{X}\right]^{-1}\mathbf{X}^T\mathbf{y}$$

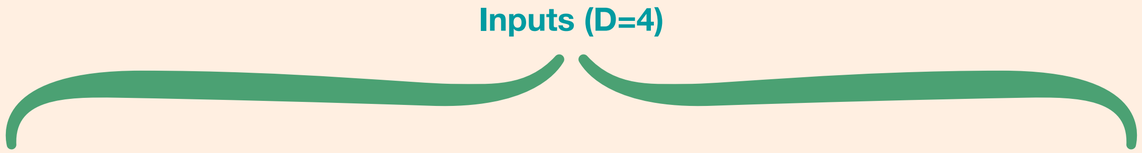
$$\hat{\boldsymbol{\beta}} = \left[\mathbf{X}^T\mathbf{X}\right]^{-1}\mathbf{X}^T\mathbf{y}$$

OLS estimate for the parameters of a linear model

What if there are multiple inputs?

Matrix math makes it easy to generalize equations for an arbitrary number of parameters

As long as the design matrix has as many columns as there are free parameters, the math still holds



Output

Sepal.Length <dbl>	Sepal.Width <dbl>	Petal.Length <dbl>	Petal.Width <dbl>	age <dbl>
5.1	3.5	1.4	0.2	15.20667
4.9	3.0	1.4	0.2	13.50667
4.7	3.2	1.3	0.2	14.02667
4.6	3.1	1.5	0.2	13.38667
5.0	3.6	1.4	0.2	15.40667
5.4	3.9	1.7	0.4	15.94667
4.6	3.4	1.4	0.3	14.20667
5.0	3.4	1.5	0.2	14.68667
4.4	2.9	1.4	0.2	12.70667
4.9	3.1	1.5	0.1	13.88667

1–10 of 150 rows

Previous 1 2 3 4 5 6 ... 15 Next

With 4 input features, the model will need to learn 5 total parameters

design matrix will have one column for the intercept, and one for each input

```
as.matrix(c(1,2,3,4,5))
```

just some random values for illustration

```
     [,1]
[1,]    1
[2,]    2
[3,]    3
[4,]    4
[5,]    5
```

β

```
model.matrix(age ~ Sepal.Length + Sepal.Width + Petal.Length + Petal.Width,data=iris_age)
```

	(Intercept)	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
1	1	5.1	3.5	1.4	0.2
2	1	4.9	3.0	1.4	0.2
3	1	4.7	3.2	1.3	0.2
4	1	4.6	3.1	1.5	0.2
5	1	5.0	3.6	1.4	0.2
6	1	5.4	3.9	1.7	0.4
7	1	4.6	3.4	1.4	0.3
8	1	5.0	3.4	1.5	0.2
9	1	4.4	2.9	1.4	0.2
10	1	4.9	3.1	1.5	0.1

$\hat{X}$

`as.matrix()` **turns a list into a column vector**

`as.vector()` **returns a row vector**

this is important to keep in mind so that the dimensions match

In R, you can perform matrix multiplication with the %*% operator

make predictions by taking the product between the design matrix and parameter vector

```
X <- model.matrix(age ~ Sepal.Length + Sepal.Width + Petal.Length + Petal.Width,data=iris_age)
B <- as.matrix(c(1,2,3,4,5))
X %*% B
```

```
    [,1]
1   28.3
2   26.4
3   26.2
4   26.5
5   28.4
6   32.3
7   27.5
8   28.2
9   25.1
10  26.6
```

μ

There are $N = 150$ rows, one for each training sample for which we made a prediction with this random beta vector

The OLS estimate of the parameters minimizes SSE…but does not provide any confidence intervals